

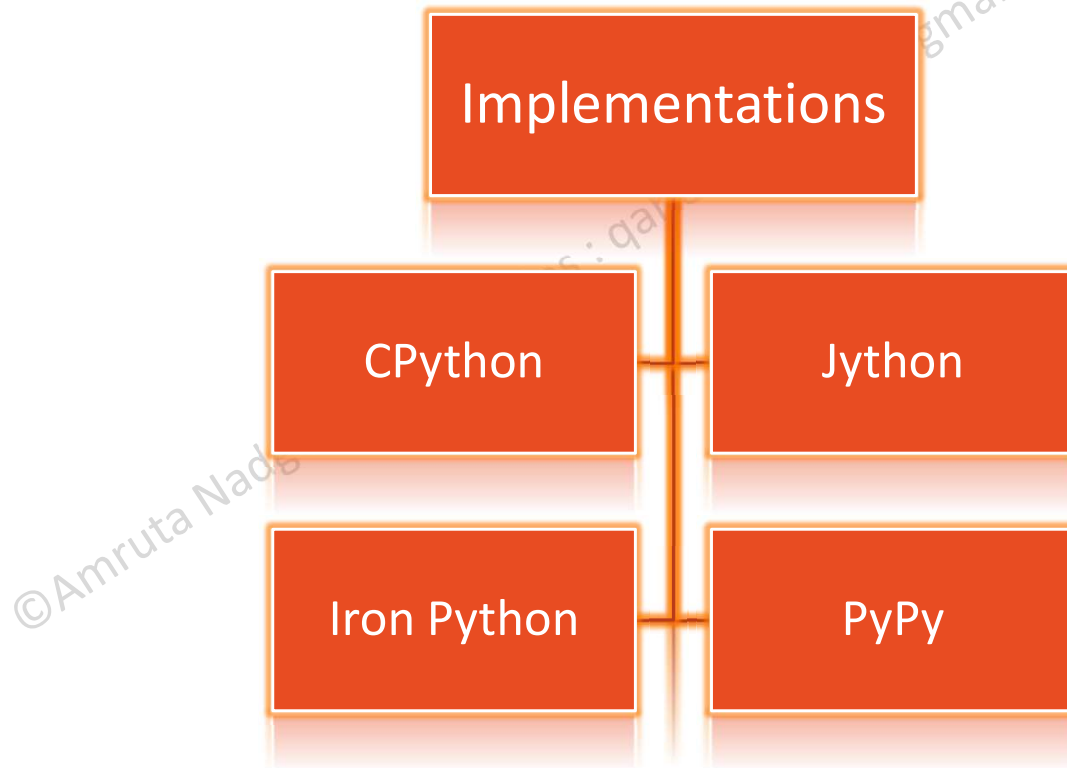
Python - Introduction

©Amruta Nadgunde queries : qahelp.amruta@gmail.com

Introduction

- Python developed by Guido van Rossum, is a general-purpose programming language
- Python is very high level, dynamic, object-oriented and cross functional language
- Can be used in just about any area of software development
- The python standard library supplies many well-designed extension modules that can be reused effectively
- Can be used for tasks as representing data, processing text, interacting with the operating system and filesystem, and web programming
- These modules help python code access functionality provided by underlying OS, Database, network or GUI
- Effective in performing computationally intensive tasks such as XML parsing and numeric array computations

Implementation



Installation

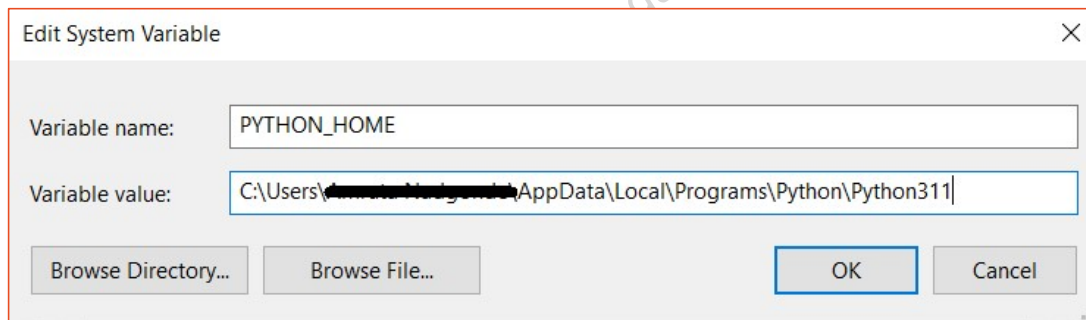
- Visit <https://www.python.org/downloads/>, select Download python



- Once the download is complete, start installation and proceed as mentioned the installation guide

Setting up environment variables

- The default path of python installation is :
C:\Users\<<>>\AppData\Local\Programs\Python\PythonXXX
- Copy this path and set a new system environment variable – PYTHON_HOME



- Edit the path variable as shown

%PYTHON_HOME%
%PYTHON_HOME%\Scripts

Installing Package

- Python has a package manager – PIP
- A package contains all the files you need for a module [code libraries]
- To download a package - Open the command line interface -> navigate to the python script folder [C:\Users\<>\AppData\Local\Programs\Python\Pythonxxx\Scripts]
- Give command `pip install <package name>`
- Package will be available in
C:\Users\<>\AppData\Local\Programs\Python\Python311\Lib\site-packages
- You can also check it using `pip show <package_name>` / `pip list`

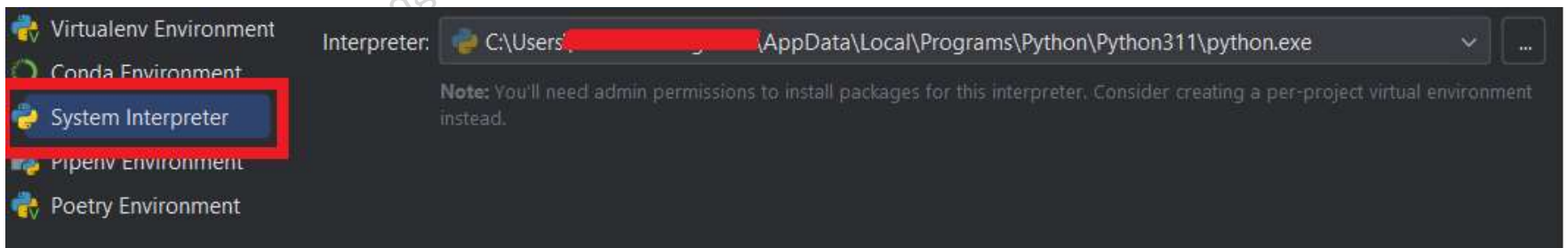
IDE for Python

- Navigate to <https://www.jetbrains.com/pycharm/download/#section=windows>
- Community version -> click on it -> download exe
- Install the editor by as per the installation instructions
- Open PyCharm -> Create new project -> Navigate to your workspace folder -> Name your project -> Create
- This creates a new Project

©Amruta Nadgunde

Setting up the interpreter

- Click on the right bottom of the window -> Select Interpreter Settings
- In Settings window -> Click on Add Interpreter -> Add local interpreter
- Click on System Interpreter
- Give path of your Python interpreter as shown below



Python REPL

- The Python standard shell, or REPL (Read-Eval-Print Loop), allows you to run Python code interactively
- Easy to debug or test your code
- The Python interpreter can execute Python code in two modes – Script / REPL
- REPL (Interactive) stands for Read-Evaluate-Print-Loop
- Input is read from the keyboard and output is printed on terminal (screen)
- The input consists of Python code, which the interpreter parses and evaluates
- Run 'python' command from your cmd to start python REPL
- To open script in interactive mode run `python -i <file.py>` from current dir

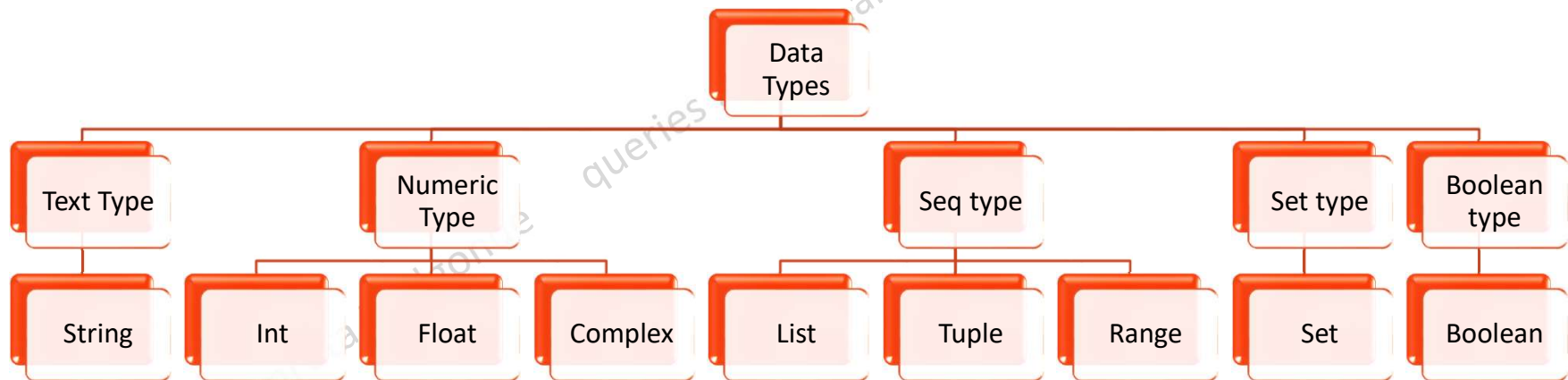
Language Basics

©Amruta Nadgondkar
Queries : qahelp.amruta@gmail.com

Data Type

- Python is dynamically typed language, data type is not mandatory while declaring a variable
- `a = 10` is a valid statement in python, where an int variable 'a' is assigned value 10
- Data values in python are known as Objects. `type()` determines which operations can be performed on object and also its attributes
- The built-in fundamental data types include numbers, strings, lists, dictionaries, tuples and sets
- Python supports integers, floating point and complex numbers
- Strings in python can be enclosed in single (' ') or double (" ") quotes

Python Data Types



Python Keywords

- Keywords are the reserved words that have special meaning and purpose
- These keywords are always available and do not need any import explicitly
- Python 3 has 35 keywords
- All the Python IDEs will highlight keywords to differentiate them from other words in your code
- You can use `help("keywords")` function to get list of keywords in python REPL
- Using `help("<keyword_name>")` will give more information about that keyword
- The `kwlist` property of keyword module will also print list of keywords
- The `iskeyword()` function will check if given string is a keyword

Keyword Type	Keyword	Keyword Type	Keyword
Value	True	Control Flow	if
	False		elif
	None		else
Operator	and	Iteration	for
	or		while
	not		break
	in		continue
	is		else
Structure	def	Returning	return
	class		yield
	with	Exception Handling	try
	as		except
	pass		raise
import	lambda		finally
	import		else
	from		assert
	as		

Conditions

- The if statement in python is written by using 'if' keyword, followed by a condition to be evaluated
- If clause terminates with a :
- Indentation is mandatory after if statement while writing the computation block
- An if can be optionally followed by an 'else' clause or elif clause
- 'break' is used to terminate the loop
- 'continue' is used to stop the current iteration and move to the next one

For loop

- The for loop in python is used to iterate over collection of data like lists, tuples, sets and dictionaries
- The for loop does not require an indexing variable to be set in advance
- Most commonly it is used to visit each item of the collections
- Since 'strings' are iterable objects (sequence of bytes) , the for loop can be used to iterate over the strings also
- The range() function returns a sequence of numbers, starting from 0 by default, and increments by 1 (by default), and ends at a specified number
- 'break' and 'continue' statements can be used in for loop just like if statement
- An else statement can be given with loop, which executes after the loop finishes

While loop

- With the while loop we can execute a set of statements as long as a condition is true
- While loop requires relevant variable to be ready, which is set to a starting value and either incremented or decremented
- 'break' and 'continue' statements can be used in while loop just like if statement

```
i = 0
```

```
while i < 10:
```

```
    print(i)
```

```
    i += 1
```

- An else statement can be given with loop, which executes after the loop finishes
- Else is executed if break is not executed

Operators

➤ Following operators are available in python

- Arithmetic operators
- Assignment operators
- Comparison operators
- Logical operators
- Identity operators
- Membership operators
- Bitwise operator

Arithmetic Operator

Operator	Name
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus
**	Exponentiation
//	Floor division

Assignment Operators

Operator	Name
=	Assign a value to a variable
+=	Add and assign value in result
-=	Subtract and assign value in result
*=	Multiply and assign value in result
/=	Divide and assign value in result

Comparison Operators

Operator	Name
==	Equal
!=	Not equal
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

Logical, Identity and Membership Operator

Operator	Description
and	Returns True if both statements are true
or	Returns True if one of the statements is true
not	Reverse the result, returns False if the result is true
is	Returns True if both variables are the same object
is not	Returns True if both variables are not the same object
in	Returns True if a sequence with the specified value is present in the object
not in	Returns True if a sequence with the specified value is not present in the object

In-built Functions and Types

print	Print objects to the text stream file. If file object is None, sys.stdout will be used
input	Read a line from input, convert it to a string and return it
type	With one argument, return the type of an object.
chr	Return the string representing a character whose Unicode code point is the integer arg
ord	Given a string representing one Unicode character, return an integer representing the Unicode code point of that character
len	Return the length (the number of items) of an object
str	Built-in string class
int	Return an integer object constructed from a number or string arg
help	Invoke the built-in help system
dir	Return the list of names in the current local scope With an argument, attempt to return a list of valid attributes for that object.

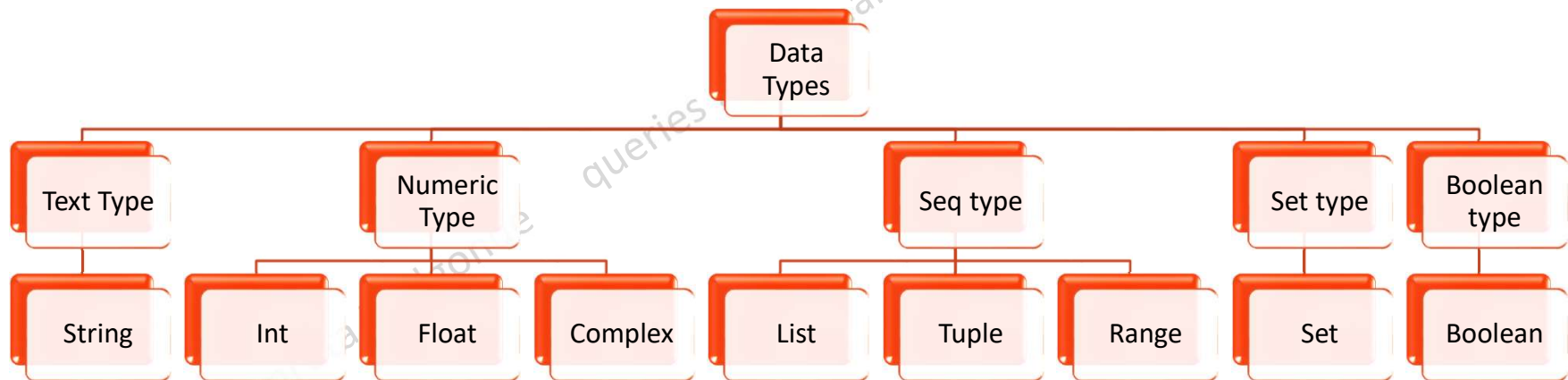
Data Types

©Amruta Nadgondkar
Queries : qahelp.amruta@gmail.com

Data Type

- Python is dynamically typed language, data type is not mandatory while declaring a variable
- `a = 10` is a valid statement in python, where an int variable 'a' is assigned value 10
- Data values in python are known as Objects. `type()` determines which operations can be performed on object and also its attributes
- The built-in fundamental data types include numbers, strings, lists, dictionaries, tuples and sets
- Python supports integers, floating point and complex numbers
- Strings in python can be enclosed in single (' ') or double (" ") quotes

Python Data Types



Strings

- Python strings can be enclosed in single or double quotes
- You can assign a multiline string to a variable by using three quotes
- Strings in python are arrays of bytes, which can be iterated using for loop
- Strings are immutable
- The string characters can be accessed using an index in square brackets
- Presence of a substring inside a string can be found by 'in' operator
- To check if certain phrase is not a part of string we can use 'not in' operator
- Strings can be concatenated by using '+' sign

String – Indexing and Slicing

- Strings are ordered sequences of character data.
- Individual characters in a string can be accessed by specifying the string name followed by a number in square brackets
- String indices can also be specified with negative numbers, in which case indexing occurs from the end of the string backward
- Python allows to extract substrings from a string, known as string slicing.
- If `s` is a string, an expression of the form `s[m:n]` returns the portion of `s` starting with position `m`, and up to but not including position `n`
- For any string `s` and any integer `n` ($0 \leq n \leq \text{len}(s)$), `s[:n] + s[n:]` will be equal to `s`
- The slice method also accepts stride (also called a step), which indicates how many characters to jump after retrieving each character in the slice

Method	Description	Example
isalpha()	Returns True if a string contains characters that are classified as a letter in the Unicode character database	ABCabc', 'aab', 'BBC' - Allowed 'ABC123', 'ab c' - not allowed
isdecimal()	Returns true if a character is an integer in the Base-10 number system i.e. set of positive integers {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}	0', '110' - allowed '3.142', '-99' - not allowed
isdigit()	Returns true if a symbol is an expression of the ten decimal numbers Unicode subscripts, superscripts are allowed	\u00B2 (superscript 2) - allowed Negative vlaues - not allowed
isnumeric()	Returns True if a symbol can be conceptually interpreted as a number Roman numerals, fractions, currency numerators etc are allowed	\u2168(Roman 9), \u0024 (dollar sign) - allowed Negative values- not allowed
isalnum()	Returns true for all four conditions above	\u00B2', 'abc123', '11zz', '12345'
isprintable()	Returns true if all characters in the string are printable	Everthing except newlines, carriage returns, separators
isidentifier()	Returns true if a string is a valid name for a Python object String must follow rules for naming objects	hello', 'h_e_l_l_o' - allowed '11hello' - not allowed

String Methods – Case Conversion

Method	Description
<code>capitalize()</code>	Returns a copy of <code>s</code> with the first character converted to uppercase and all other characters converted to lowercase
<code>lower()</code>	Returns a copy of <code>s</code> with all alphabetic characters converted to lowercase
<code>swapcase()</code>	Returns a copy of <code>s</code> with uppercase alphabetic characters converted to lowercase and vice versa:
<code>title()</code>	Returns a copy of <code>s</code> in which the first letter of each word is converted to uppercase and remaining letters are lowercase
<code>upper()</code>	Returns a copy of <code>s</code> with all alphabetic characters converted to uppercase

String Methods

Method	Description
count()	Counts occurrences of a substring in the target string
find()	Searches the target string for a given substring.
strip()	Removes any whitespace from the beginning or the end
replace()	Replaces a string with another string
split()	Returns a list where the text between the specified separator becomes the list items
format()	Takes the passed arguments, formats them, and places them in the string where the placeholders {} are
partition()	Searches for a specified string, and splits the string into a tuple containing three elements
Join()	Concatenates strings from an iterable.

Data Type Comparison

DESCRIPTION	ORDERED	ALLOW DUPLICATES	MUTABLE	SYNTAX	EXAMPLES
List	✓	✓	✓	[]	• [1, 2.3, True] • ['John', 'Doe']
Tuple	✓	✓	✗	()	• ('age', 22) • (7.89, False)
Set	✗	✗	✓	{ }	• {6, 4.5} • {'nice', True}
Dictionary	✓	✗	✓	{key: value}	• {"FB": "Facebook", "WA": "WhatsApp", "IG": "Instagram"} • {'name': 'Bob', 'id': 255}

Lists

- Lists in python are used to store collection of data
- A list can have a nested list stored inside it
- List can store mixed data in an ordered form [index starting with 0]
- List allows duplicate values and are mutable
- The len() function is used to find length of the list
- Items of list can be accessed using their index – lst[0] gives first item in list lst
- For loop can be used to iterate over the list collection
- The 'in' operator is used to check presence of an item in the list

List Methods

Method	Description
append	Adds an element at the end of the list
extend	Add the elements of any iterable at the end of the list
index	Returns the index of the first element with the specified value
insert	Adds an element at the specified position
remove	Removes the item with the specified value
reverse	Reverses the order of the list
sort	Sorts the list

List Comprehension

- Used to create lists in a concise way, eliminating need for a loop
- Syntax :
 [expression for item in list if condition == True]
- For every item in list, execute the expression if the condition is True.
- List comprehensions can utilize conditional statements like if...else to filter existing lists
- We can also use list comprehension with iterables other than lists

Tuples

- Tuples, like lists are used to save collection of data
- Items in the tuple are ordered but unchangeable
- Tuple data is written in a round bracket ()
- Tuples allow duplicates
- Tuple items can be accessed using index given in square bracket []- ordered
- For loop can be used to iterate over the tuple collection
- The 'in' operator is used to check presence of an item in the tuple
- Once a tuple is created it is immutable; to mutate a tuple you first need to convert it into a list update it and convert back to tuple

Sets

- Sets, like lists are used to save collection of data
- Items in the set are unordered and unchangeable
- Since add/remove is possible, Sets are mutable
- Set data is written in a curly bracket {}
- Sets do not allow duplicates
- For loop can be used to iterate over the set collection
- An item can be added in the set using add() method and removed using remove() method
- The 'in' operator is used to check presence of an item in the set

Set Methods

Method	Description
add	Adds an element in the set
remove	Removes the item with the specified value, throws error if not present
discard	Removes an item from the set if it is present
union	Creates a new set containing elements of both the sets excluding duplicates
intersection	Creates a new set containing common elements of both the sets
difference	Creates a new set containing items from first set but not the second
symmetric_difference	Creates a new set containing elements of both sets excluding the common ones

Dictionaries

- Dictionaries in python are used to store key:value pairs
- Items in the dictionary are ordered, changeable and do not allow duplicate keys
- Dictionary data is written in a curly bracket {} and has key:value pairs
- Dictionary can also be created using dict() function
- The dictionary items can be access by their keys given in square brackets []
- keys() method is used to get list of keys
- Unhashable objects (like list) can not be used as dictionary keys
- For loop can be used to iterate over the dictionary

Dictionary Methods

Method	Description
fromkeys()	Returns a dictionary with the specified keys and value
get()	Returns the value of the specified key
items()	Returns a list containing a tuple for each key value pair
keys()	Returns a list containing the dictionary's keys
pop()	Removes the element with the specified key
popitem()	Removes the last inserted key-value pair
setdefault()	Returns the value of the specified key. If the key does not exist: inserts the key, with the specified value
update()	Updates the dictionary with the specified key-value pairs
values()	Returns a list of all the values in the dictionary

Functions & Modules

©Amruta Nadgunde
queries : qahelp.amruta@gmail.com

Python Functions

- Function : self contained block of code that encapsulates a specific task or related group of tasks
- Benefits :
 - Abstraction of Functionality
 - Modularity
 - Namespace separation
- Types of Arguments :
 - Positional Arguments
 - Keyword Arguments
 - Default Parameters
 - Variable arguments (*args, **kwargs)

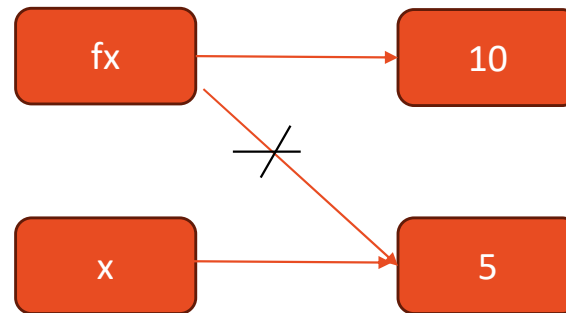
Argument Passing in Python

- Everything in python is an Object

`x = 10` : Creates `x` as a reference that refers to object with value 10

```
def foo(fx):  
    fx = fx * 2
```

```
x = 5  
foo(x)  
Print(x)
```



Python Namespace

- Used to organize symbolic names and objects that they refer to
- They are like dictionary where name is a key and actual object is a value
- Built-in : Contains names of all python built-in objects. Available all the time while python is running
- Global : Contains names of all the objects for currently running program as well as objects of imported modules
- Local : For each function a new local namespace is created containing all the objects of the function
- Enclosing : Contains names of all the objects of enclosing function

Variable Scope

- Multiple instances of same name can exist in different namespaces simultaneously
- Python will determine the name being referred to using scope of the variable
- Scope : Region in the program where name has a meaning
- The order in which variables are resolved is Local -> Enclosing -> Global -> Built-in
- Namespace dictionaries are used to implement namespaces
- Functions `globals()` and `locals()` can be used to access the global and local namespace respectively

Python Modules

- Modules help in Modularity, reusability, simplicity and maintainability
- Module : A file with legitimate python code with .py extension
- Modules may contain one or more reusable functions
- Python standard library contains lots of built-in python modules
- The Python Package Index (PyPi) provides a centralized repository for third-party Python modules on the Internet
- Each module has its own private symbol table – global symbol table for all its object (thus each module has a separate namespace)

Import Statement

- In order to use objects of a different module into your current one, you need to import the module – e.g. `import <module_name>`
- When a module is imported only module name (not its objects) is placed in the callers symbol table, hence to access the objects from the imported module you need to access using a ‘.’
- `from <module_name> import <names (s)>` : This allows individual objects to be imported directly into caller’s private symbol table
- Import all is not recommended as it might accidentally overwrite existing names
- `from <module_name> import <name> as <alt_name>` : Import names into caller's symbol table with an alternate name to avoid conflicts
- try with `ImportError` can be used to guard against unsuccessful attempts of imports

Module vs Script

- `__name__` is a special variable in python, which is set by the interpreter to indicate current context of a module
- When a .py file is imported as a module, python sets var `__name__` as name of the module
- However when .py file is executed as script , `__name__` is set to a string with value `__main__`
- This can be used to differentiate between module and script
- `__name__ == '__main__'` block is commonly used to define code that should only be executed when the file is run as a script, not when imported as a module

Python Packages

- When large applications are to be built modules may grow in numbers, managing them can become a problem
- Python packages help in organizing the modules
- Packages allow for a hierarchical structuring of the module namespace using dot notation
- Packages also help avoid collisions between module names.
- The import statement is used to access the packages and modules within them

```
import <pck_name.module_name>[, <pck_name.module_name> ...]
```

```
from <package_name.module_name> import <name(s)>
```

```
from <package_name> import <module_name> as <alt_name>
```

Functional programming

- An imperative code focuses on writing step by step commands describe HOW a program should behave
- A declarative approach on the other hand focuses on specifying the result of WHAT you want
- To support functional programming a language should :
 - Take another function as an argument
 - Return another function to it's caller
- In python you can assign a function to a variable and then use the variable as function itself
- Function can be passed as an argument to another function or function can be return another function
- When a python function is passed as an argument it is called as a call-back function, wherein the callback function can modify the behavior of called function

Python Lambdas

- Lambda is used to define an anonymous function – a shorthand option in place of regular function
- Syntax : `lambda <param_list> : <expression>`
- The value of lambda is a callback function. It takes arguments as specified by `<param_list>` and returns a value as indicated by `<expression>`
- Rules
 - In a lambda function, statements like `return`, `pass`, `assert`, or `raise` are not allowed
 - Lambda allows single expression
 - Lambda can return only single value
 - Python lambda expressions support all the different ways of passing arguments

Map(), Filter() and Reduce()

- `map()`: with `map()` you can apply a function on every element of an iterable and `map` will return an new iterator

`map(<f>, <iterable>)`

- `filter()`: allows you to filter items from an iterable based on evaluation of the given function

`filter(<f>, <iterable>)`

- `reduce()`: applies a function to the items in an iterable two at a time, progressively combining them to produce a single result

`from functools import reduce`

`reduce(<f>, <iterable>)`

Python Decorators

- Python decorators are used to modify the behavior of a python function/method without actually modifying any code
- Functions in python are first class objects and hence they can be assigned to a variable, or passed as a parameter to another function
- Functions can be defined inside another function, such functions are called inner functions
- A decorator takes a function (e.g. func) as an argument and returns a new function (e.g. wrapper)
- When decorator is applied, wrapper function is executed and then original inner function is called
- Put simply: decorators wrap a function, modifying its behavior.

Python Decorators

```
def my_decorator(func):  
    def wrapper(*args, **kwargs):  
        print('Function started')  
        func(*args, **kwargs)  
        print('Function ended')
```

```
    return wrapper
```

```
my_func = my_decorator(my_func)
```

```
my_func
```

Python Generators

- Python generators are special type of functions
- These functions use 'yield' keyword and return an iterator object
- 'yield' produces a series of results over time, this allows the function to pause its execution after each yield, rather than computing and storing all values at once
- Useful in creating functions that are memory-efficient, especially when dealing with large or infinite sequences.
- Can be useful while
 - Reading large files line by line.
 - Generating infinite sequences (e.g., Fibonacci numbers).
 - Processing data streams efficiently.

Python Object Oriented

©Amruta Nadgunde : qahelp.amruta@gmail.com

Python Classes

- Keyword 'class' is used to create a Class
- A special method `__init__` is used for object initialization
- Methods within the class are defined in the same way as that of function using `def` keyword
- Each method must be provided with a special parameter called 'self' as the first parameter (this is like pointer in C++ or this keyword of java)
- The value of the self parameter is provided by python
- Instance variables can be made private by prefix '`__`'
- The objects are created by assigning a call to the class name to each of the variables

```
mycar = Car()
```

Class Syntax

#class starts with class keyword followed by name and

class Class_Name:

 #class attributes – always public, accessed using class modifier

 var = value

 #__init__ method used to initialize object attributes

 def __init__(self, attri1, attri2):

 self._attri1 = attri1

 self._attri2 = attri2

 def method1(self, [param list]):

 //method body

The '.__init__' Method

- To construct an object of any class, you need to call class with appropriate arguments
- In python classes, __init__ method is used for object attribute initialization
- Class constructors internally trigger the instantiation process -
 - instance creation - Python classes have a special method called .__new__(), which is responsible for creating and returning a new empty object.
 - instance initialization – Takes new object as the first argument (self), sets the required attributes to a valid state based on the arguments passed to it
- The class constructor accepts the same arguments as the .__init__() method
- .__init__() must not explicitly return anything different from None
- If your subclasses provide a .__init__() method, then this method must explicitly call the base class's .__init__() with appropriate arguments

The 'self' keyword

- Each instance method in python takes 'self' as the first parameter
- The self keyword is used to represent an instance (object) of the given class
- Self allows each instance of a class to have its own attributes and behavior
- Even long before creating these objects, we reference the objects as self while defining the class.
- In python whenever we call an instance method, the object reference is implicitly passed to the method call

```
emp = Employee(100, 'aaa')
```

```
emp.printDeatils() -> Employee.printDetails(emp, 100,'aaa')
```

Object Creation

#Object Creation

obj1 = Class_Name()

obj1.method1() #method call

#Accessing class attributes

obj1.__class__.var

©Amruta Nadgunde queries : qahelp.amruta@gmail.com

Naming Conventions

- Python does not distinguish between public, private or protected like most of the languages do
- However, Python has a well-established naming convention to communicate the intended usage of a variable [e.g. a variable **should be** used as a private member]
- Public members are declared using the normal naming pattern. [empid, name]
- Private members are declared using a leading underscore [_empid, _name]
- This decision will depend on how you want your users to use your classes
- Names with two leading underscores triggers – Name Mangling
- This results in name hiding – i.e. mangled names aren't available for direct access
- Name mangling is particularly useful when you want to ensure that a given attribute or method won't get accidentally overwritten

Class vs Instance Attributes

- **Class attribute** - a variable that you define in the class body directly.
- Class attributes belong to their containing class. Their data is common to the class and all its instances.
- **Instance attributes** - a variable that you define inside a method.
- Instance attributes belong to an Object of a given class. Their data is only available to that instance and defines its state.
- Class attributes should be accessed using `classname.<attribute>`
- Even though you can define instance attributes inside any instance method, it's best to define all of them in the `.__init__()` method, which is the instance initializer
- You can't access instance attributes through the class. You need to access them through their objects only

The `.__dict__` Attribute

- Both classes and instances have a special attribute called `.__dict__`.
- It holds a dictionary containing attributes and methods of underlying class
- Each key in `.__dict__` represents an attribute name. The value associated with a given key represents the value of the corresponding attribute.
- When you access a class member through the class, object/instance member through object, Python automatically searches for the member's name in the class `.__dict__`/in the instance `.__dict__`
- If the name is not there you get 'AttributeError' or 'NameError' respectively

Methods in the Class

- Python allows you to define three type of methods –
 - Instance methods, which take the current instance, self, as their first argument
 - Class methods, which take the current class, cls, as their first argument
 - Static methods, which take neither the class nor the instance
- The self argument holds a reference to the current instance, allowing you to access that instance from within methods
- Static methods do not take instance or class as an argument.
- A good practice is to define a static method instead of a regular function outside the class when that function is closely related to your class
- You should generally call static methods through the corresponding class instead of one of its instances.

Python Special Methods

- Python supports some special methods also known as magic or 'dunder' methods [as they have **double underscores**]
- Python calls them automatically in response to specific operations
- `__init__()` is the most commonly used dunder method which is used for object initialization
- The `__str__()` method provides what's known as 'the informal string representation' of an object.
- This string representation can be used either by `str()` or `print()` function
- `__repr__` is used to provide an unambiguous string representation of an object.
- The `__repr__` method gets called when we call `repr()` function
- This method should return a string that can be used to recreate the object

Python Properties

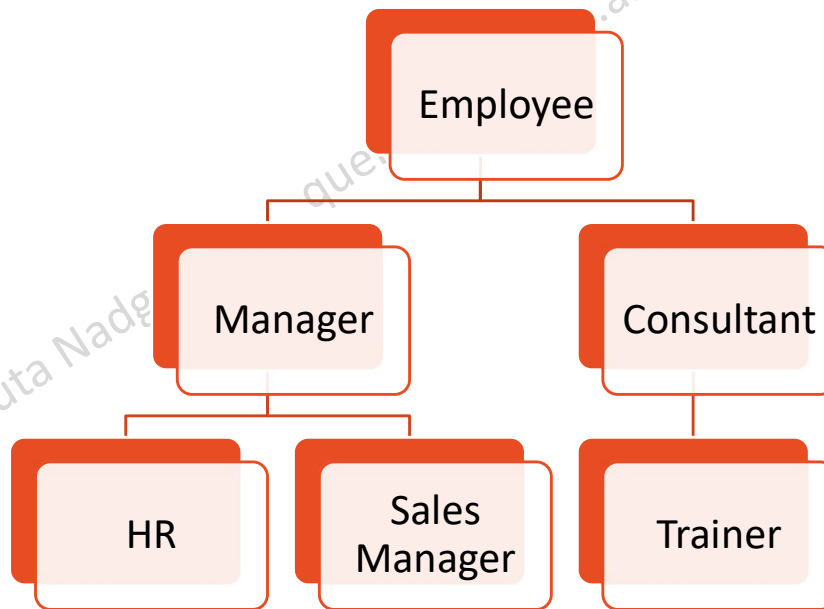
- Encapsulation is one of the fundamental principle of OOPs
- Languages like C++ or Java make extensive use of getter and setter methods to access or mutate the attributes
- Using methods to access and update attributes promotes encapsulation, thereby protecting an object's state or data from the outside world
- However In Python, it's completely normal to expose attributes as part of an object's public API
- Python uses properties to add function like behavior on top of your attributes
- Accessing attributes directly from outside the class is a common practice in python

Python Properties

- To turn an existing attribute like name into a property, you typically use the `@property` decorator to write the getter method
- The getter method must return value of the attribute , which is stored in non public `_name` attribute
- To define the setter method of a property-based attribute, you need to use the decorator `@attr_name.setter`
- Property setters should accept an argument that will provide an value you are trying to set
- Inside the setter method you can do the value check using conditions / process the value before it is set to the non public `_name` attribute

Inheritance

- Process by which an object can acquire properties of other object.
- “is-a” a kind of hierarchy.

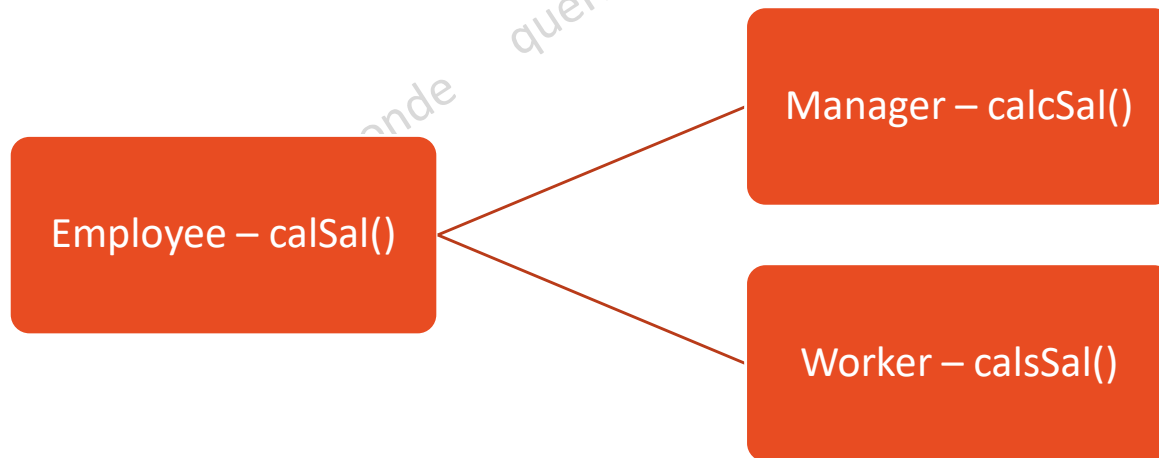


Inheritance

- Inheritance is a mechanism to create hierarchy of multiple related classes
- Python is one of the few languages that support multiple inheritance (more than one base classes)
- To make a Python class inherit from another, you need to list the parent class's name in parentheses after the child class's name in the definition
- The related classes share a common interface as defined by the base class
- Super keyword is used to refer to the base class implementation. `super().__init()` can be used for super class member initialization
- Classes in hierarchy support polymorphism – method overriding
- Python supports multi level as well as multiple inheritance

Polymorphism

- The ability of different type of related objects to respond to the same message in their own ways.
- Poly -> Many and Morph -> Form
- One command can invoke different implementation for different related objects.



Abstract Class

- Abstract Class: A class which contain generic/common features that multiple derived classes can share.
- An abstract method is a method that is declared without an implementation
- Abstract classes are created to support polymorphism and are usually present at the top of the hierarchy
- Abstract classes can not be instantiated
- Python provides the abc module to define abstract base classes. Leading underscores can be used to communicate that its an abstract class
- The abc module in the Python standard library provides functionality to prevent creating objects from abstract base classes

More on Inheritance

- When derived classes inherits a base class it inherits both the base class interface as well as implementation
- Most of the modern languages do not allow inheriting from multiple classes, rather they allow you to implement multiple interface
- In python explicit interface declaration is not needed - Any object that implements the desired interface can be used in place of another object.
- However, inheritance still gives an advantage as derived classes can leverage most of their base class implementation
- Inheritance strictly displays is-a relation [customer may have id and name , but is not an employee]

Exception Handling

©Amruta Nadgunde queries : qahelp.amruta@gmail.com

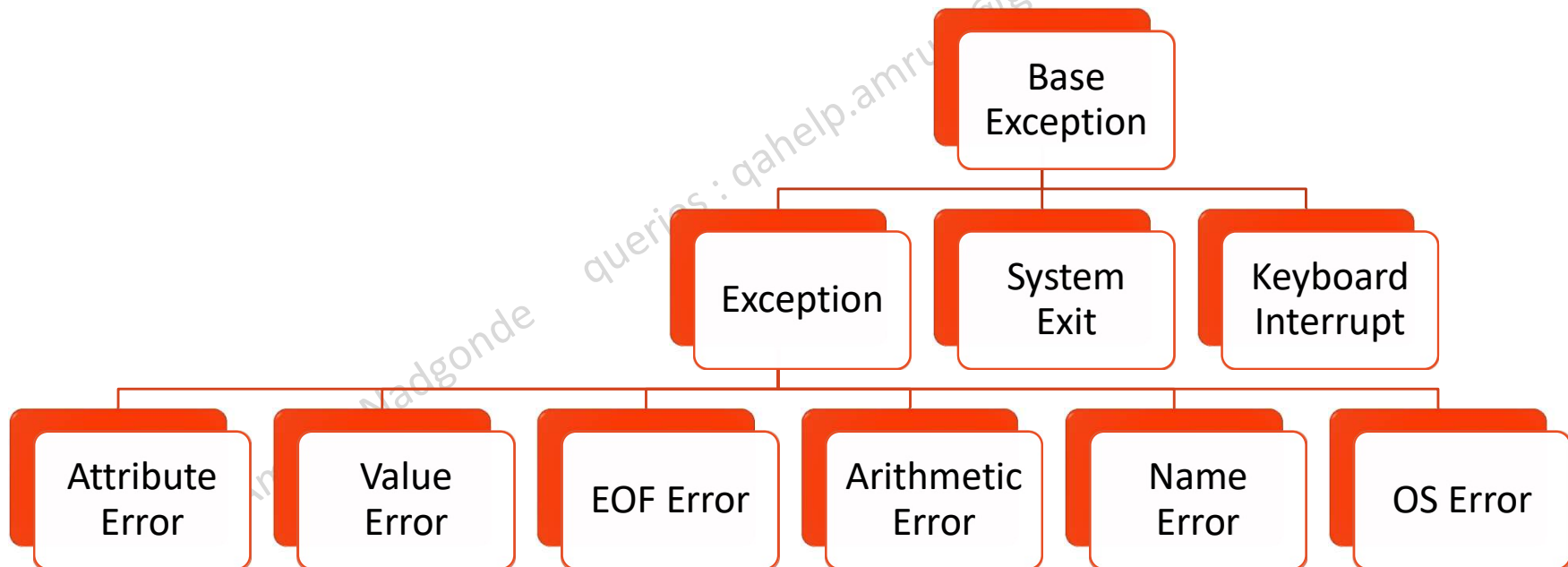
Errors

- Programs cannot be 100% error-free
- Logical errors:
 - Occur when there is something wrong with the logic used by the programmer.
 - Can be corrected by debugging and dry runs.
- Run Time errors
 - Occur at run-time, when the program is executed.
 - Can be controlled by predicting the error and checking appropriate condition.

Exception Handling in Python

- In languages like C there is no separate mechanism for error handling, it rather relies on returning a number as an error code
- Business logic gets secondary importance whereas error handling is given primary importance
- Python has a structured way of handling the exceptions
- The try and except block in Python is used to catch and handle exceptions
- The code block that may run into an error is put into the try block whereas the except statement is the program's response to any exceptions in the preceding try clause
- Every raised exception has a traceback (stack trace) - a report containing the sequence of calls and operations that traces down to the current exception
- The traceback header is Traceback (most recent call last) , then the actual call stack and the exception name followed by its error message

Exception Hierarchy



Exception Keywords

- try: the code that might run into an exception is placed in the try block
- except: The error handling mechanism is placed in the except block
- The generic except block is not recommended, you should identify the issues in the program before hand and add specific exception classes to handle the errors
- If more than one class of error is expected in the code, multiple except blocks can be added to address each such error
- raise: An exception [both in-built/custom] can be explicitly raised (unlike thrown) using raise keyword
- else: The code in else block will get executed after try block executes successfully
- finally: The code in finally block will get executed always

Custom Exceptions

- You should raise exceptions that clearly communicate the problem you're dealing with
- Custom exceptions are those that you create when no built-in exception fits your needs
- While defining a custom exception you must inherit from another exception class, typically Exception
- In Python, defining custom exceptions with a pass statement as the class body is a common practice, class name is often the most important thing
- However, you can add code in this class as per your need

©Amruta Nandwade Queries : qandelp.amruta@gmail.com

Logging

- This module defines functions and classes which implement a flexible event logging system for applications and libraries
- Logs are useful in debugging process
- The `logging.basicConfig()` method in Python's logging module provides a convenient way to set up a basic logging configuration for your application
- This method configures the "root logger," which is the default logger used by any logging calls
- Here, `level`: Sets the minimum logging level to be handled, `filename` can be provided if logs need to be stored in the file
- `format`: Specifies the format string for log messages. This uses standard `LogRecord` attributes like `%(levelname)s`, `%(asctime)s`, `%(message)s`

Log Levels

Level	Information
DEBUG (10)	Detailed information, typically useful only when diagnosing problems.
INFO (20)	Confirmation that things are working as expected
WARNING (30)	An indication that something unexpected happened
ERROR (40)	Defines serious problems
CRITICAL (50)	A severe error, indicating that the program itself may be unable to continue running

NumPy

©Amruta Nadgunde

queries : qahelp.amruta@gmail.com

Introduction

- NumPy is a third-party Python library that provides support for multidimensional arrays and functions that operate on these arrays
- This is the foundation on which almost all the power of Python's data science toolkit is built
- The benefits of NumPy include:
 - More speed: NumPy uses algorithms written in C that complete in nanoseconds rather than seconds
 - Fewer loops: NumPy helps you to reduce loops and avoid errors of bounds/ indices
 - Clearer code: Without loops, your code will look more like the equations you're trying to calculate.
- NumPy is the de facto standard for multidimensional arrays in Python data science, and many of the most popular libraries are built on top of it.

Ndarray

- NumPy's basic object is the multidimensional array which is an object of ndarray class
- The array() method is used to create an array object, and it accepts iterable (tuple/list or list of list as arguments)
- The ndim attribute is used to fetch the array dimensions
- ndmin parameter is used to create higher dimension arrays
- Multidimensional arrays are like arrays within an array
- The dimensions are called as axes – for a 2D array the axes 0 would be vertical axes representing rows while axes 1 will be horizontal axes representing columns
- The axes play a major role while indexing, slicing or performing operations

NumPy Use Cases

- scalar-scalar operations ($1 + 2$)
- scalar-vector operations ($2 + \text{np.array}([1, 2])$)
- vector-vector operations ($\text{np.array}([\dots]) + \text{np.array}([\dots])$)
- vector-matrix operations [higher dimension arrays]
- matrix-matrix operations
- vectorized math ($\text{np.sin}(\text{np.array}([\dots]))$)
- Other things like dot products, cross products, inverses of matrices, root finding and fitting algorithms, polynomial functions, basis functions, etc

NumPy properties and methods

Property/Method	Description
<code>array()</code>	Creates an array from a regular Python list or tuple
<code>zeros()</code>	Creates an array full of zeros
<code>ones()</code>	Creates an array full of ones
<code>empty()</code>	Creates an array whose initial content is random and depends on the state of the memory
<code>arange()</code>	A Function which is analogous to the Python built-in range that returns an array
<code>shape()</code>	Returns a tuple of integers indicating the size of the array in each dimension
<code>dtype</code>	Returns an object describing the type of the elements in the array
<code>size()</code>	Gives total no of elements in the array, which is product of integers given by shape

Array Indexing & Slicing

- In NumPy arrays, axes are zero-indexed and identify which dimension is which
- Single element indexing works exactly like that for other standard Python sequences.
- Fewer indices than dimensions for a multidimensional array results into getting a sub dimensional array
- After indexing one gets a view – i.e. not a copy of the original, but points to the same values in memory as does the original array
- Numpy also supports advanced (integer, Boolean) indexing, where copy of array is return
- Integer array indexing allows selection of arbitrary items in the array based on their N-dimensional index
- NumPy array slicing is used to extract sub-arrays or subsets of data.

Universal Functions

- A universal function (or ufunc for short) is a function that operates on ndarrays in an element-by-element fashion
- **Vectorization** : process of performing the same operation in the same way for each element in an array without using a for loop
- Functions include standard trigonometric functions, functions for arithmetic operations, handling complex numbers, statistical functions, etc
- Numpy provides sorting (sort()) and filtering of an array.
- A filter condition is a Boolean condition that can be used to filter out elements and create a new array

Broadcasting

- **Broadcasting** : process of extending two arrays of different shapes and figuring out how to perform a vectorized calculation between them
- Broadcasting describes how NumPy treats arrays with different shapes during arithmetic operations
- The smaller array is “broadcast” across the larger array so that they have compatible shapes
- Broadcasting Rules:
 - Makes the two arrays have the same number of dimensions. If the numbers of dimensions of the two arrays are different, adds new dimensions with size 1 to the head of the array with the smaller dimension
 - Makes each dimension of the two arrays the same size. If the sizes of each dimension of the two arrays do not match, dimensions with size 1 are stretched to the size of the other array.

Universal Functions

Function	Description
min, max	returns minimum or maximum of an array or along an axis
ptp	returns range of values (maximum-minimum) of an array or along an axis
percentile(a, p, axis)	calculate pth percentile of array or along specified axis
median	compute median of data along specified axis
mean	compute mean of data along specified axis
std	compute standard deviation of data along specified axis
var	compute variance of data along specified axis
average	compute average of data along specified axis

Universal Functions

Function	Description
sin, cos, tan	compute sine, cosine and tangent of angles
arcsin, arccos, arctan	calculate inverse sine, cosine and tangent
hypot	calculate hypotenuse of given right triangle
sinh, cosh, tanh	compute hyperbolic sine, cosine and tangent
arcsinh, arccosh, arctanh	compute inverse hyperbolic sine, cosine and tangent
deg2rad	convert degree into radians
rad2deg	convert radians into degree

Pandas

©Amruta Nadgunde queries : qahelp.amruta@gmail.com

Introduction

- Pandas is a Python library written in C, which is mainly used for cleaning, analyzing, exploring and manipulating data
- Pandas allow analysing Big data and make conclusions based on statistical theories like min, max, average, correlation between columns etc
- The fundamental objects of Pandas lib are Series and DataFrame
- Series : series is a one dimensional array that can hold any data types
- DataFrame: Fundamental Pandas object, which is 2 dimensional size-mutable, potentially heterogenous tabular data structure

Series

- A Series object wraps two components, viz a sequence of values and sequence of indices (default starts at 0)
- These components are accessed with `.values` and `.index`, respectively
- A series can be created using scalar data like List/tuple , key-value pairs like dictionaries or ndarray
- Series can hold only one attribute per key (data with 2 columns)
- RangeIndex – Positional implicit index that starts with 0 and indicates row index
- Labelled : Explicit index which is a hashable object like string/number
- Method `loc[]` is used to access labelled index while `iloc[]` is used to access range index
- One can also use negative indices and slices

DataFrame

- Series can hold only one attribute per key, real data can have multiple attributes and span many columns
- Dataframe is a fundamental object of Pandas, created by data from list of lists, dictionaries, 2D numpy array or data from CSV/JSON
- Series objects can be combined into a DataFrame by providing a dictionary in the constructor
- The 2 dimensions of a DataFrame can be referred to as axes – where axis[0] is the row index (vertical) and axis[1] is the column index (horizontal)
- A DataFrame is also a dictionary-like data structure, so it also supports .keys() – which returns the set of column headers

DataFrame Methods

Method/Attribute	Description
shape	Displays no of elements in each dimension
info()	Displays all columns and their data types
describe()	Displayes some basic descriptive statistics for all numeric columns
head()	Fetches first 5 rows from the recordset
tail()	Fetches last 5 rows from the recordset
dropna()	Remove all the rows with missing values

Accessing data in DataFrame

- The `loc[]` function is labeled based data selection function
- It access a group of rows and columns by label(s) or a boolean array
- It can accept a single label (labeled index, if not set then positional index), a list or array of labels or a slice object with labels
- `Loc[]` may also accept a boolean array, where boolean array should have same length as the axis being sliced
- `iloc[]` function is used for purely index-based selection of data
- It accepts an integer (range index), an array or list of integers or slice object with int for both rows and columns

Querying & Filtering Data

- To filter data, one needs to create a filter that returns a Boolean condition which works on each row
- This condition can be sent to `loc[]` function to get filtered rows
- Rows can be selected from the DataFrame based on the values in the columns using certain queries `[df[df.Salary > 40000]]`
- Values of string type can also be used in query using string methods – e.g. `df[df.first_name.str.startswith('A')]`
- Logical operators like `or (|)` or `and (&)` can be used to create complex queries
- Statistical methods like `min`, `max`, `sum` etc can be performed on Series data
- Each column of a DataFrame is in fact a series so these methods can be performed on DataFrame column data

Pandas DataFrame Functions

Function	Description
apply	Applies a function along an axis of the DataFrame
map	Applies a function to a Dataframe elementwise
replace	Replaces values of the Series/DataFrame with other values dynamically
concat	Concatenate pandas objects along a particular axis
drop	Drop specified labels from rows or columns.
groupby	Groups DataFrame using a mapper or by a Series of columns.
value_counts	Returns a Series containing counts of unique values
unique	Returns unique values based on a hash table
count	Counts non-NA cells for each column or row
agg	Aggregates using one or more operations over the specified axis

Pandas DataFrame Functions

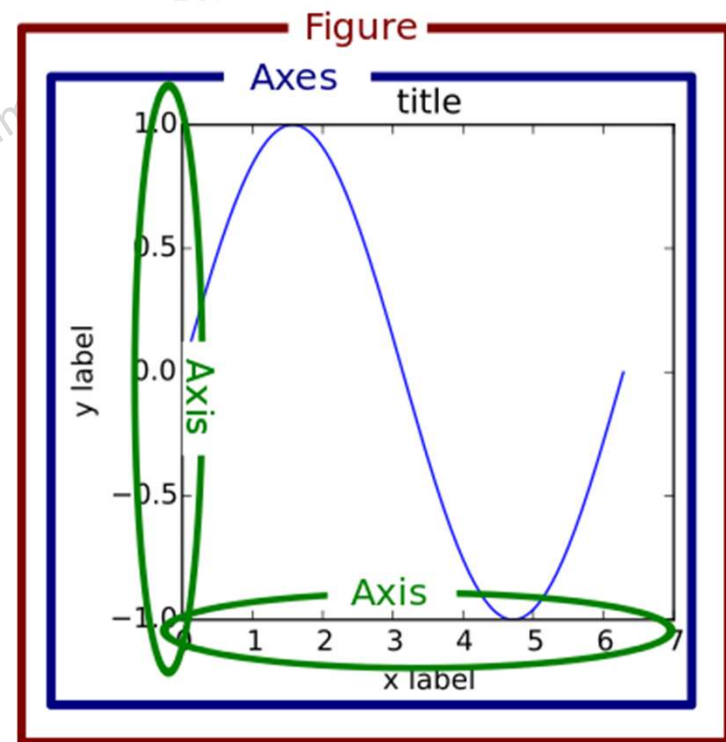
mean	Returns the mean of the values over the requested axis
median	Returns the median of the values over the requested axis.
min	Returns the minimum of the values over the requested axis
max	Returns the maximum of the values over the requested axis
sort_values	Sorts by the values along either axis by given column or list of columns
nlargest	Returns the first n rows ordered by columns in descending order
nsmallest	Returns the first n rows ordered by columns in ascending order
dropna	Removes missing values in requested axis
fillna	Fills NA/NaN values using the specified method
isna	Returns a boolean same-sized object indicating if the values are NA

Matplotlib

©Amruta Nadgunde
queries : qahelp.amruta@gmail.com

Matplotlib

- Matplotlib is a visualization library
- It is built on top of NumPy consists of several plots like line, bar, scatter, histogram, etc
- A Figure object is the outermost container for a matplotlib graphic
- A Figure contains multiple Axes objects – which represent individual plot or graph
- Below the Axes are smaller objects such as tick marks, individual lines, legends, and text boxes



Matplotlib Functions

- Plots help to understand trends, and patterns, and to make correlations
- Pyplot is a Matplotlib module that provides a MATLAB-like interface
- Axes allow placement of plots at any location in the figure, `axes([left, bottom, width, height])` is used to create the axes object
- The `plot()` function is used to plot the data along the axis
- The `show()` function is used to display the plot
- The `xlabel('X-axis')` and `ylabel('Y-axis')` functions name the x-axis and y-axis respectively
- A legend is an area describing the elements of the graph. The `legend()` function is used to place a legend on the axes
- The `title()` function is used to give title to the plot

Line Charts

- Graphical representation of data points connected by straight lines on a two-dimensional coordinate system
- A line chart can be drawn using `plot(x,y)` function, where `x`, `y` are ndarrays representing points on x-axis and y-axis respectively
- To plot only the markers, one can use shortcut string notation parameter `'o'`
- The keyword argument `linestyle`, or shorter `'ls'` can be used to change the style of the plotted line (e.g. `ls= '-'`)
- The keyword argument `color` or the shorter `'c'` can be used to set the color of the line
- The keyword argument `linewidth` or the shorter `'lw'` can be used to change the width of the line
- Every `plot()` function adds one line

Interpretation of Line Plot

Upward slope	Increasing trend
Downward slope	Decreasing trend
Sudden spike/drop	Anomaly or seasonal change
Flat line	No change

Markers

Marker	Description
'o'	Circle
'*'	Star
'.'	Point
','	Pixel
'x'	X
'X'	X (filled)
'+'	Plus
'p'	Plus (filled)
's'	Square
'D'	Diamond

Line/Colour Syntax	Description
'-'	Solid line
'.'	Dotted line
'--'	Dashed line
'-.'	Dashed/dotted line
'r'	Red
'g'	Green
'b'	Blue
'c'	Cyan
'm'	Magenta

Bar Charts

- The `bar()` function is used to draw bar charts
- The `bar(x,height)` function takes arguments that describe layout of the bars. `x`: indicate the bar's x-coordinates, `height` indicates the height of the bars is determined by the y-coordinates
- `barh()` function can be used to draw horizontal bar charts
- The keyword argument `width` is used to set the bar width, `color` is used to set bar color
- Multiple bar charts can be created in the same plot for co-relation
- Label can be added using `label` keyword argument and displayed using `legend()` function

Pie Charts

- The `pie(value)` function is used to draw pie charts, where `value` is the ndarray
- The pie chart draws one piece (called a wedge) for each value in the array
- Labels can be added to the pie chart using `label` keyword argument, where the `label` parameter is an array with one label for each wedge
- The `start angle` parameter is defined with an angle in degrees, can be change the start angle
- The `explode` parameter is used to mark one of the wedges as stand out wedge. The `explode` parameter must be an array with one value for each wedge.
- Shadow can be added to the pie chart by setting the `shadows` parameter to `True`
- Color of each wedge can be set with the `colors` parameter. Color parameter must be an array with one value for each wedge

Plots Use Cases

Line Plot	• Showing trends over time (e.g., stock prices, temperature) • Visualizing continuous data
Bar Plot	• Comparing quantities across categories • Showing ranking or categorical distribution
Scatter Plot	• Showing relationships/correlations between two numeric variables • Visualizing clustering or patterns
Pie Chart	• Showing relationships/correlations between categories • Used to show proportion/percentage of parts in a whole
Histogram	• Understanding the distribution (frequency) of a single variable • Detecting skew, modality, and spread

Seaborn

©Amruta Nadgunde queries : qahelp.amruta@gmail.com

Introduction

- Seaborn is a powerful Python library designed for creating statistical graphics
- Built on top of matplotlib, it integrates closely with Pandas data structure
- A popular choice for data visualization and exploratory data analysis in data science and machine learning workflows
- Offers a high-level, dataset-oriented API that simplifies the process of creating complex and informative statistical plots with fewer lines of code
- Provides visually appealing default styles and color palettes, enhancing the aesthetic quality of plots
- Some of the popular plots include distribution plots (histograms, kde plots), relational plots (scatter plots, line plots), Matrix plots (Heatmap) , Regression plot (regplot) etc

Types of Plots

Plot Type	Use Case	Function
Bar plot	Compare means across categories	<code>sns.barplot()</code>
Line plot	Trends over time	<code>sns.lineplot()</code>
Histogram	Distribution frequency	<code>sns.histplot()</code>
KDE plot	Smooth distribution shape	<code>sns.kdeplot()</code>
Scatter plot	Numeric relationships	<code>sns.scatterplot()</code>
Heatmap	Correlation/Matrix visualization	<code>sns.heatmap()</code>
Pair plot	Pairwise relationships in a DataFrame	<code>sns.pairplot()</code>

Distribution Plots

- Useful for visualizing the distribution of data, both univariate (single variable) and bivariate (two variables)
- One of the popular distribution plot is the histogram
- Histogram is a bar plot where the axis representing the data variable is divided into a set of discrete bins and the count of observations falling within each bin is shown using the height of the corresponding bar
- The choice of bins for computing and plotting can have substantial influence on the insights one is able to draw from the visualization
- If the bins are too large, they may erase important features, if bins are too small may be dominated by random variability, obscuring the shape of the true underlying distribution

Interpretation of Histograms

Bell-shaped	• Normal distribution • Data is evenly distributed around the mean, with most values clustering near the center
Skewed	• Uneven distribution with more values clustered on one side of the mean • Right skew : a long tail towards higher values • Left skew: a tail towards lower values.
Bimodal	• Two distinct peaks or modes • May indicate the data comes from two different underlying distributions
Uniform	• All bars are roughly the same height • Indicates that all values within the data range have a similar frequency

Kernel Density Estimation Plots

- A kernel density estimation allows us to estimate the probability density function from our finite dataset
- A probability density function describes the likelihood of a continuous random variables falling within a specific range of values, providing a relative likelihood for any given value
- A KDE plot is built by starting with finite set of data and adding a kernel centered around the data point
- Eventually all these kernels are added together to get the kernel density estimation
- KDE smooths the jagged bars of a histogram; They act as a smoothed histogram that shows where data points are concentrated.
- By estimating the probability density function, KDE plots help reveal patterns, modes, and outliers in the data

Interpretation of KDE Plots

Peaks (Modes)	• Peaks in the curve : where the data is most concentrated • A taller peak : more values are clustered around that point
Spread (Width of the Curve)	• Wide curves : more variability in data • Narrow curves : more consistent values.
Number of Peaks	• One peak : unimodal distribution • Two or more : multimodal distribution [Subgroups]

Tall peak	Many values around this point
Wide base	More spread/variation
Multiple peaks	Possible subgroups
Flat, wide curve	Uniformly spread data

Line Plots

- A type of data visualization used to display the relationship between two variables, typically showing trends over time [or something ordered]
- Particularly effective for visualizing continuous data where the order of observations matters, such as time series data or data with a clear ordinal relationship between variables
- The relationship between x and y can be shown for different subsets of the data using the hue, size, and style parameters
- The line plot calculates and displays a 95% confidence interval using bootstrapping
- Bootstrapping is a resampling technique that estimates the sampling distribution of a statistic by repeatedly drawing samples with replacement from the observed data.
- The shaded region represents the range within which the true mean or regression line is likely to fall. A wider band indicates greater uncertainty

Interpretation of Line Plot

Upward slope	Increasing trend
Downward slope	Decreasing trend
Sudden spike/drop	Anomaly or seasonal change
Flat line	No change
Gaps between lines (with hue)	Difference between groups

Scatter Plots

- A type of statistical plot used to visualize the relationship between two numerical variables
- Each data point is represented as a marker on a two-dimensional plane, with its position determined by the values of the two variables
- Scatter plots effectively show the correlation, direction (positive or negative), and strength of the relationship between variables
- The primary semantic properties of scatter plot include
 - Hue : Maps a variable to the color of the points, enabling differentiation of groups.
 - Size : Maps a variable to the size of the points, representing magnitude
 - Style : Maps a variable to the marker style (e.g., circles, squares, triangles), providing another layer of visual distinction

Interpretation of Scatter Plots

Diagonal up	Positive correlation
Diagonal down	Negative correlation
Spread out randomly	No correlation
Clusters	Subgroups present
Isolated points	Outliers or rare cases

Heatmap

- A Seaborn heatmap is a graphical representation of two-dimensional data where individual values within a matrix are represented by colors
- Particularly effective for displaying correlation matrices or identifying patterns across two variables.
- Heatmaps are widely used to display correlation matrices, showing the strength and direction of relationships between features in a dataset
- Seaborn offers various parameters to customize heatmaps, including color palettes, annotations and control over the color bar
- Help in identifying trends, clusters, and anomalies within the data by observing how colors change across the axes

Interpretation of Scatter Plots

Correlation Value	Meaning
1	Perfect positive correlation
-1	Perfect negative correlation
0	No linear relationship
~0.7 to 1	Strong positive correlation
~0.3 to 0.7	Moderate positive correlation
~0 to 0.3	Weak or no correlation

Color	Meaning
Red	Negative correlation
Blue	Postive correlation
White/Neutral	Near 0 (no correlation)

Plotly

©Amruta Nadgunde queries : qahelp.amruta@gmail.com

Plotly

- Plotly is a Python library used to create interactive plots like line charts, bar charts, scatter plots, heatmaps
- The two main interfaces provided to create plots :
 - `plotly.graph_objects`
 - `plotly.express`
- `plotly.graph_objects` provides low level API which gives fine-grained control over every aspect of the plot.
- Using this, one can manually create Figure, Trace, Layout etc
- `plotly.express` provides much simpler, high-level API which is built for quick prototyping using pandas DataFrames
- It automatically handles axes, legends, colors, etc.

Plotly Graphs

Plot Type	Function	Description
Line plot	<code>px.line()</code>	Line chart for trends over time
Scatter plot	<code>px.scatter()</code>	Basic scatter plot
Bubble chart	<code>px.scatter()</code>	Scatter with size-based markers
Bar chart	<code>px.bar()</code>	Vertical/horizontal bar chart
Histogram	<code>px.histogram()</code>	Frequency distribution
Box plot	<code>px.box()</code>	Statistical summary (min, Q1, median, etc.)
Violin plot	<code>px.violin()</code>	Kernel density estimation + boxplot combo

Plotly Graphs

Plot Type	Function	Description
Density heatmap	<code>px.density_heatmap()</code>	2D histogram with color as frequency
ECDF plot	<code>px.ecdf()</code>	Empirical Cumulative Distribution Function
Sunburst	<code>px.sunburst()</code>	Hierarchical chart with radial layout
Treemap	<code>px.treemap()</code>	Hierarchical chart with nested rectangles
Pie chart	<code>px.pie()</code>	Basic pie chart
Funnel chart	<code>px.funnel()</code>	Stages in a process

Seaborn vs Plotly

- Seaborn is an excellent choice for static plots where as Plotly can create great interactive plots
- Seaborn usually give fast performance when using large datasets
- Seaborn is useful for Reports, PDFs, static images, where as Plotly is for Dashboards, web apps, data exploration
- Plotly provides tools such as Hover tooltips, Zoom & pan, Click-based filtering, 3D plots
- Use Seaborn if you want quick, beautiful static plots such as when you are preparing reports or academic papers.
- Use Plotly if you need interactive dashboards or web apps
- Plotly is also useful for data exploration and engages users with hover, zoom, and filters

Beautiful Soup

©Amruta Nadgondkar
ques : qahelp.amruta@gmail.com

Introduction

- Python library designed for parsing HTML and XML documents
- Creates parse trees that helps to extract data from HTML documents you've scraped from the internet
- "Web scraping" usually refers to a process that involves automation
- Challenges :
 - Variety : Every website has different structure
 - Durability : The structure of the website keep changing

Finding Elements

- `find()` and `find_all()` methods are used to search and extract specific HTML elements from a parsed document
- The `find()` method returns first matching tag, while `find_all()` method returns all matching tags
- Common Parameters for Both:

Parameter	Description
<code>name</code>	Tag name to search for (e.g. 'div', 'p')
<code>attrs</code>	Dictionary of tag attributes (e.g. {'class': 'header'})
<code>string</code>	Search for tags containing specific string

Methods

Method / Property	Description	Example
<code>find()</code>	Finds the first matching tag	<code>soup.find('p')</code>
<code>find_all()</code>	Finds all matching tags (returns list)	<code>soup.find_all('a')</code>
<code>select()</code>	Finds tags using CSS selectors	<code>soup.select('div.class')</code>
<code>select_one()</code>	Finds first tag using CSS selector	<code>soup.select_one('#id')</code>
<code>get_text()</code>	Gets an inner text of a tag	<code>tag.get_text()</code>
<code>text</code>	Gets string content of a tag	<code>soup.p.text</code>
<code>name</code>	Returns the tag name	<code>tag.name</code>
<code>attrs</code>	Dictionary of all attributes	<code>tag.attrs</code>
<code>tag['attr']</code>	Access a specific attribute	<code>tag['href']</code>
<code>string</code>	Inner string of a tag	<code>tag.string</code>

Methods

Method / Property	Description	Example
parent	Gets the parent of a tag	tag.parent
parents	Gets all ancestors of a tag	list(tag.parents)
children	Generator of direct children	list(tag.children)
contents	List of direct children	tag.contents
descendants	Generator of all nested children	list(tag.descendants)
next_sibling	Gets the next sibling tag	tag.next_sibling
previous_sibling	Gets the previous sibling tag	tag.previous_sibling
next_element	Next element in the tree	tag.next_element
previous_element	Previous element in the tree	tag.previous_element
clear()	Clears contents inside tag	tag.clear()

Pillow

IMAGE PROCESSING

©Amruta Gadgonde queries : qahelp.amruta@gmail.com

Introduction

- A fork of PIL (Python Image Library) that adds powerful image processing capabilities to python programs
- Features:
 - Load and Save images
 - Crop, resize, reduce, rotate and flip images
 - Change between color modes (RGB, CMYK, Grayscale)
 - Split and merge channels (RGB)
 - Apply filters
- Pillow makes image handling easy and scriptable, which is useful in Image processing, web apps, automation tools, fun projects etc

Image Class Methods

<code>Image.open(path)</code>	Opens an existing image file
<code>save(path, format=None)</code>	Saves the image to a file
<code>load()</code>	Allocates storage for the image and loads the pixel data
<code>show()</code>	Opens the image using the default viewer
<code>copy()</code>	Returns a copy of the image
<code>resize(width, height)</code>	Resizes the image
<code>crop(box)</code>	Crops a region from the image, box - tuple(left,top, right, bottom)
<code>rotate(angle)</code>	Rotates the image
<code>transpose(method)</code>	Flips or rotates the image
<code>convert(mode)</code>	Converts to a different color mode
<code>split()</code>	Splits into separate channels (R, G, B)
<code>merge(mode, bands)</code>	Combines channels into an image
<code>filter(ImageFilter.FILTER_NAME)</code>	Applies a filter (e.g. BLUR)

Image Filtering

- The ImageFilter module in the Pillow library provides a set of built-in filters
- These filters can be used with the filter() method of an Image object to perform image processing effects such as blurring, sharpening, and edge detection
- Most of these filters are implemented using convolution kernels behind the scenes.
- A convolution kernel is a small matrix (e.g. 3×3) used to apply effects on the images through a mathematical operation called convolution.
- Convolution involves sliding the kernel over each pixel of the image and multiplying the values in each of the kernel's cells with the corresponding pixel values in the image
- The results are added together to produce the new pixel value
- The process is repeated for every pixel in the image moving the kernel each time so that the kernel's central cell corresponds to a different image pixel each time.

Image Filters

ImageFilter.BLUR	Applies a basic blur
ImageFilter.CONTOUR	Detects edges and outlines
ImageFilter.DETAIL	Enhances detail
ImageFilter.EDGE_ENHANCE	Enhances edges
ImageFilter.EDGE_ENHANCE_MORE	Stronger edge enhancement
ImageFilter.EMBOSS	Embossed 3D effect
ImageFilter.FIND_EDGES	Highlights edges
ImageFilter.SMOOTH	Smooths the image
ImageFilter.SMOOTH_MORE	Stronger smoothing
ImageFilter.SHARPEN	Sharpens the image

Image Bands and Modes

- Image is a 2D array of pixel, where each pixel can be represented by different color channel depending on the mode
- In an RGB image, each pixel is represented by three values corresponding to the red, green, and blue values for that pixel
- The Image object of RGB image contains 3 bands , one for each color
- An Image with size 100*100 is represented by a 100*100 * 3 array of values [RGB]
- Pillow supports working with standard modes like RGB, RGBA, Grayscale, CMYK
- The split() method in Pillow (PIL) is used to separate an image into its individual channels, where each channel represents the intensity of that color only.
- The channels can be recombined using merge() method

Database

©Amruta Nadgunde queries : qahelp.amruta@gmail.com

Data Storage using files

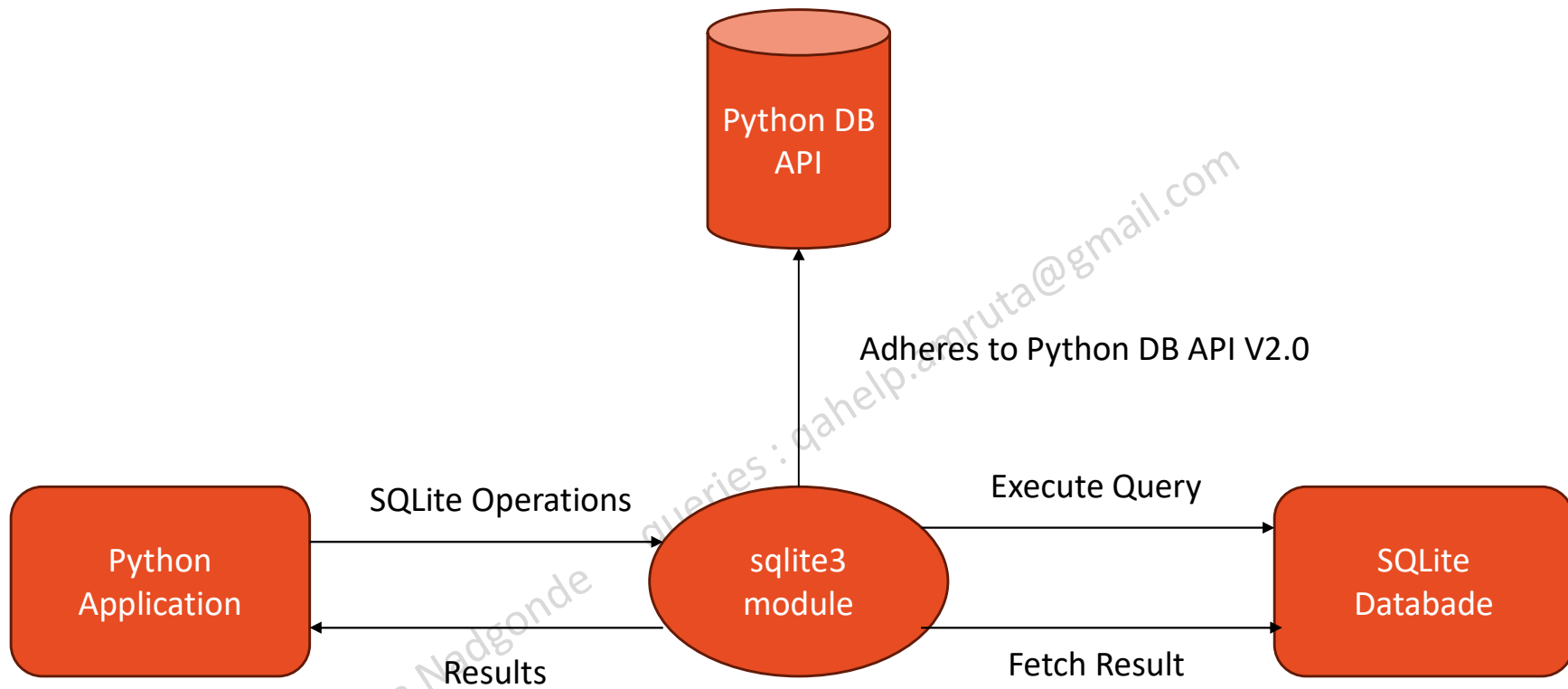
- A flat file is a file containing data with no internal hierarchy and usually no references to external files
- Since the field widths are not fixed, other formats like CSV are often used
- Being able to save data in files help you to share information between users and sites, and also to maintain state of an application
- In order to read/process data from a file an application must need to understand the structure of the data
- Working with flat files is manageable, straightforward to implement and most of the languages have in built parsers to work with the files
- However, managing data in files become difficult as the volume of data increases

Data Storage using RDBMS

- Relational databases provide a way to store structured data in tables and establish relationships between those tables
- Normalization : The way data is organized in tables that the result of database operations is always unambiguous and as intended
- RDBMS use SQL as a primary language to interact with data
- An SQLite database provides support for interacting with the data table using SQL
- SQL is a declarative language used to create, manage, and query the data contained in a database
- Conceptually data is stored in two-dimensional table format, Each table consists of rows of records, and each record consists of columns, or fields containing data

SQLite3

- By default, your Python installation contains a Python SQL library named `sqlite3` that you can use to interact with an SQLite database.
- SQLite databases are serverless and self-contained, since they read and write data to a file, you don't even need to install and run an SQLite server to perform database operations
- SQLite comes built in with most computers, mobiles and browsers
- Python's `sqlite3` module adheres to python DB API specification V2.0(PEP249)
- PEP249 provides SQL interface to encourage and maintain simplicity between modules that are used to access database



Database connectivity

While interacting with database we need to perform following 7 steps

1. Import the package
2. Load and register the driver
3. Establish a connection
4. Associate SQL with the connection
5. Execute the query
6. Process the result
7. Close the connection

Working with DB

- Make a connection : `connect()` method of `sqlite3` module will establish a connection with DB at given path. It will return a Connection object
- Execute queries : To execute queries in SQLite, use `cursor.execute()` method. `execute()` can execute any query sent to it as a string
- A Cursor object represents a database cursor which is used to execute SQL statements, acts like a middleware between SQLite database connection and SQL query
- Connection -> `commit()` : Commits any pending transaction to the database
- Cursor -> `fetchall()` : Returns all /remaining rows of a query result as a List
- Cursor -> `close()` : Closes the cursor object

MySQL

- MySQL is an open-source DB which quickly became market leader in DB solutions
- MySQL comes with a script that allows you to set the password security level, assign admin passwords, and add and remove user account privileges
- A database driver is a piece of software that allows an application to connect and interact with a database system.
- These drivers obtained as third-party modules, comply and implement the Python Database API standard interface [[PEP 249](#)]
- To interact with MySQL database using Python one needs to install Python MySQL connector
- The most popular one is MySQL Connector/Python [mysql-connector-python] can be installed using pip installer

MySQL Connection

- The connect() method from mysql.connector module is used to establish connection to MySQL database
- To this function one needs to send parameters like hostname, username ,password and db name. This returns Connection object
- 'cursor' objects make use of a Connection object to interact with your MySQL server. To create a cursor, use the .cursor() method of your connection variable
- The execute() method of the cursor object can be execute the query sent to it as string, while the commit() method makes the changes persistent
- To execute a DQL query one needs to use fetchall() method, This method fetches all rows from the last executed statement